

- JS2-数据类型
 - 4 参数传递
 - 4.1 基本数据类型的参数传递
 - 4.2 引用数据类型的参数传递

JS2-数据类型

4 参数传递

JavaScript 中所有参数传递都是 **值传递**，但由于数据类型的不同，表现出两种行为：

1. 基本数据类型传递的是值的副本
2. 引用数据类型传递的是引用地址的副本

4.1 基本数据类型的参数传递

当传递基本数据类型时，函数接收的是原始值的一个 **副本**，而非值本身。因此，函数内部对参数的修改不会影响外部变量。

```
function modifyValue(num) {  
    num = num + 1;  
    console.log('函数内部的 num:', num);  
}  
  
let res = 0;  
modifyValue(res);  
console.log('函数外部的 res:', res);  
  
// 函数内部的 num: 1  
// 函数外部的 res: 0
```

内存分析

1. 变量 `res` 在栈内存中存储值 `0`
2. 调用 `modifyValue(res)` 时，将 `res` 的值复制一份传递给参数 `num`
3. 参数 `num` 在函数内部有自己独立的内存空间
4. 修改 `num` 的值不会影响原始的 `res`

4.2 引用数据类型的参数传递

当传递引用数据类型时，函数接收的是引用地址的副本。虽然地址被复制，但它们指向同一个堆内存对象。因此，函数内部对参数的修改会影响外部变量。

```
function modifyArray(arr) {
  arr.push(4);
  console.log('函数内部的 arr:', arr);
}

let myArray = [1, 2, 3];
modifyArray(myArray);
console.log('函数外部的 myArray:', myArray);

// 函数内部的 arr: [ 1, 2, 3, 4 ]
// 函数外部的 myArray: [ 1, 2, 3, 4 ]
```

内存分析

1. 变量 myArray 在栈内存中存储引用地址（如 0x123），指向堆内存中的数组 [1, 2, 3]
2. 调用 modifyArray(myArray) 时，将引用地址 0x123 复制一份传递给参数 arr
3. 参数 arr 和原始 myArray 指向同一个堆内存对象
4. 修改 arr 会直接影响堆内存中的对象，因此外部的 myArray 也会改变

许多人认为 JavaScript 中引用数据类型是 "引用传递"，这是不准确的。实际上：

- 引用数据类型传递的是引用地址的副本
- 虽然地址被复制，但它们指向同一个对象
- 因此，修改对象属性会影响外部变量，但 **重新赋值** 不会

```
function modifyReference(obj) {
  obj = { x: 100 }; // 重新赋值，改变引用地址
  console.log('函数内部的 obj:', obj);
}

let originalObj = { x: 1 };
modifyReference(originalObj);
console.log('函数外部的 originalObj:', originalObj);

// 函数内部的 obj: { x: 100 }
// 函数外部的 originalObj: { x: 1 }
```

作者：海底火旺

链接：<https://juejin.cn/post/7508998028742721574>

来源：稀土掘金

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。